PROMPT 4 — Lógica Fake en el Teléfono (App Android)

> Documento-prompt para una IA implementadora. Debe **crear el código Java** de la lógica fake de la app Android y entregarlo compilando + pruebas automáticas en verde.
>
> **Ante cualquier duda o falta de información: PÁRATE y pregunta al responsable del proyecto ANTES de escribir o modificar nada.** No asumas ni inventes.

---

1. Rol y tarea

Eres un ingeniero de software Android (Java, compatible con el esqueleto existente `BeaconApp`).

Tarea: implementar la clase `LogicaFake` (de la sección 3, marcada `(new)`). Recibe los datos de la medición del beacon, los **transforma** y los envía por **POST** al servidor REST usando el cliente `PeticionarioREST` ya existente.

2. Contexto del proyecto

- Muchos datos llegan del **Arduino** tal cual los emite el beacon: en forma de **bytes**.
- La **app los transforma** (byte → número entero) y en el POST los manda como números (`Z/N`).
- El servidor REST espera: `POST` **con** `{"minor": 1234}` (más metadatos opcionales) y responde `200 {"ok": true, "id": N}` (Prompt 3).
- La app YA tiene `PeticionarioREST.java` (cliente HTTP, `hacerPeticionREST(metodo, url, cuerpo, callback)`) y las clases BLE. **No modifiques** `Utilidades`, `TramaIBeacon` ni `PeticionarioREST`.
- Estilo del esqueleto: comentarios en español, separadores `// ----`, logs con `Log.d(ETIQUETA_LOG, ...)`.

3. Diseño a implementar (notación: Logical Design & Reverse Engineering v3)

3.1 Clase NUEVA `LogicaFake`

```
      ------------------- LogicaFake (NEW) -------------------
      |
      | URL_SERVIDOR: Text   (const, p.ej. "http://10.0.2.2/<carpeta>/mediciones")
      |
      |
nombre_emisora: Text, uuid: Text,
major: Byte, minor: Byte, txPower: Byte,
laRespuesta: RespuestaREST
                             --> guardarMedicion() --x   // (new)
      |
      | // no devuelve nada (void): la informacion sale por el POST.
      | // laRespuesta es un callback: ahi llega la respuesta del
```

```
        | // servidor, de forma asincrona (no como valor de retorno).
        |
        ------------------------------------------------------------------
```

Los campos `nombre_emisora: Text`, `uuid: Text`, `major: Byte`, `minor: Byte`, `txPower: Byte` se **reciben en el método** (es el diseño que corresponde): así llegan de la trama del beacon. El método **no devuelve nada**: la información viaja en el POST y la confirmación vuelve por callback.

3.2 Ficha del método

Método	Firma	Comportamiento
`guardarMedicion()`	`nombre_emisora: Text, uuid: Text, major: Byte, minor: Byte, txPower: Byte, laRespuesta: RespuestaREST`	**No devuelve nada** (`void`). **Transforma los campos `Byte` a número entero (`Z`)** y construye el JSON del servidor; luego lo envía con `new PeticionarioREST().hacerPeticionREST("POST", URL_SERVIDOR, cuerpoJSON, laRespuesta)`. La respuesta llega **después**, de forma asíncrona, vía el callback `laRespuesta`: se considera éxito si `codigo == 200`.

El método NO devuelve nada: el dato se va por el POST (efecto lateral) y la confirmación del servidor vuelve por el callback `laRespuesta`, nunca como valor de retorno.

Transformación obligatoria: el valor numérico se conserva; solo cambia la representación (`Byte` → entero). Así del Arduino llega en bytes y en el POST viaja como `Z`, que es lo que espera el servidor.

Secuencia obligatoria dentro de `guardarMedicion()` (tú escribes el código):

1. Log de entrada con `Log.d` en tag `">>>>"` (estilo del esqueleto).
2. Construir en un único string un **JSON** con exactamente esta forma (los valores salen de los parámetros recibidos, ya transformados a entero, en este orden):

```
{ "minor": 1234,
  "uuid": "EPSG-GTI-PROY-3A",
  "major": 2816,
  "tx_power": 4,
  "nombre_emisora": "Minerva_ELBACON" }
```

3. Log del JSON y de la URL antes de enviar.
4. Enviarlo con el cliente HTTP ya existente: un único POST con esa URL y ese cuerpo.
5. Log final; el callback recibe `codigo == 200` si el servidor registró la medición.

> Valores de ejemplo del proyecto (los que envía el beacon): emisora `"Minerva_ELBACON"`, `tx_power 4`, uuid `"EPSG-GTI-PROY-3A"`, `major 2816`, `minor 1234`. En el código comenta, en el punto de transformación, que del Arduino llegan como bytes y se amplían a entero.

> Ruta: usa la canónica del Prompt 3, `/mediciones`, en `URL_SERVIDOR`. Con **XAMPP**, pon delante la carpeta de `htdocs` que corresponda (`http://10.0.2.2/<carpeta>/mediciones`).
>
> Recuerda (fuera de scope, avísalo): Android bloquea HTTP por defecto → config de cleartext o emulador `http://10.0.2.2/<carpeta>/mediciones` (puerto 80 con XAMPP).

4. Fichero a entregar

`BeaconApp/app/src/main/java/org/com/example/mmarvid/BeaconApp/logicafake/LogicaFake.java`

5. Requisitos de código y de comentarios (OBLIGATORIOS)

1. Comentarios **en español**, simples.
   - Cabecera de la clase: qué hace y a qué objetivo del proyecto sirve.
   - Cabecera del método (antes de la firma): `PROPOSITO` / `PARAM.` / `RETORNA` / `NOTA`.
   - Comentarios 1–2 líneas en cada paso (transformar Byte→N, construir JSON, enviar).
2. Fidelidad al estilo del esqueleto: separadores `// ----`, cierre `// ()`, tag de log `">>>>"`.

6. Pruebas automáticas (OBLIGATORIAS)

6.1 `BeaconApp/app/src/main/java/org/com/example/mmarvid/BeaconApp/logicafake/LogicaFakeTest.java` (JUnit 4, JVM, sin red)

1. `URL_SERVIDOR` **termina en** `/mediciones`.
2. `construirJSON()` (helper `protected`) recibe los 5 datos del beacon y genera una cadena única que contiene `"minor": 1234`, `"major": 2816`, `"tx_power": 4`, `"uuid": "EPSG-GTI-PROY-3A"` y `"nombre_emisora": "Minerva_ELBACON"` (comprobar con los valores de ejemplo del proyecto).
3. **Los numéricos se envían como enteros**: `construirJSON(...)` NO contiene `"minor": 1234.0` ni comillas alrededor de `1234`, `2816` o `4` (van sin comillas, como `Z/N`).

6.2 Ejecución

```
cd BeaconApp
./gradlew test
```

Resultado en verde. Documentar en un bloque `/* ===== PRUEBAS ===== */`.

7. Verificación y entrega

- [ ] Clase `LogicaFake` con `guardarMedicion(5 datos del beacon, laRespuesta)` (**sin retorno, `void`**) que transforma `Byte → entero` y monta `{"minor": 1234, ...}`.
- [ ] Se usa `PeticionarioREST` con `"POST"`; la confirmación vuelve por el callback `laRespuesta`. No se modifican las clases BLE/REST existentes.
- [ ] Comentarios en español y estilo del esqueleto. `./gradlew test` en verde.
- [ ] Resumen final: fichero creado, comandos usados y resultado.